

Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.

- It is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data in different places), which otherwise unnecessarily increases the size of the database.
- Data redundancy leads to inconsistency problems during insert, delete, and update operations, making data management difficult.
- Normalization involves splitting a table into multiple tables, which must be linked each time a query requires data from the split tables.

## Before Normalization

**Employee\_Department**

| Emp_ID | Emp_Name   | Department | Dept_Location | Emp_Skills           |
|--------|------------|------------|---------------|----------------------|
| 101    | Nick Wise  | HR         | London        | Recruitment, Payroll |
| 102    | John Cader | Finance    | Australia     | Budgeting            |
| 103    | Lily Case  | HR         | London        | Recruitment          |
| 104    | Ford Dawid | IT         | Chicago       | Programming, Testing |

## After Normalization

**Employee**

| Emp_ID | Emp_Name   | Dept_ID |
|--------|------------|---------|
| 101    | Nick Wise  | D1      |
| 102    | John Cader | D2      |
| 103    | Lily Case  | D1      |
| 104    | Ford Dawid | D3      |

**Department**

| Dept_ID | Department | Dept_Location |
|---------|------------|---------------|
| D1      | HR         | London        |
| D2      | Finance    | Australia     |
| D3      | IT         | Chicago       |

**Employee Skills**

| Emp_ID | Emp_Skills  |
|--------|-------------|
| 101    | Recruitment |
| 101    | Payroll     |
| 102    | Budgeting   |
| 103    | Recruitment |
| 104    | Programming |
| 104    | Testing     |

- **Before Normalization:** The table is prone to redundancy and anomalies (insertion, update, and deletion).
- **After Normalization:** The data is divided into logical tables to ensure consistency, avoid redundancy and remove anomalies making the database efficient and reliable.

### Problems in the Employee\_Department Relation

This section highlights common anomalies and redundancy issues that occur due to improper database design.

1. **Insertion Anomaly:** If a new department is created but no employee is assigned to it yet, we cannot store its location because we need an employee record to insert.
2. **Update Anomaly:** If the location of the HR department changes, we must update it in multiple rows (for both Nick Wise and Lily Case). If one row is missed, the data becomes inconsistent.
3. **Deletion Anomaly:** If all employees in the IT department leave, we lose the department information, including its location.
4. **Data Redundancy:** The department location is repeated for every employee in the same department.

### Need of Normalization

The primary objective for normalizing the relations is to eliminate the below anomalies. Failure to reduce anomalies results in data redundancy, which may threaten data integrity and cause additional issues as the database increases.

- **Insertion Anomalies:** Insertion anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. For example, if a database requires that every record has a primary key, but no value is provided for a particular record, it cannot be inserted into the database.
- **Deletion anomalies:** Deletion anomalies occur when deleting a record from a database and can result in the unintentional loss of data. For example, if a database contains information about customers and orders, deleting a customer record may also delete all the orders associated with that customer.
- **Updation anomalies:** Updation anomalies occur when modifying data in a database and can result in inconsistencies or errors. For example, if a database contains information about employees and their salaries, updating an employee's salary in one record but not in all related records could lead to incorrect calculations and reporting.

### Features of Database Normalization

This section highlights the key benefits of normalization in improving data organization, consistency, and overall database design.

- **Elimination of Data Redundancy:** One of the main features of normalization is to eliminate the data redundancy that can occur in a database. Data redundancy refers to the repetition of data in different parts of the database. Normalization helps in reducing or eliminating this redundancy, which can improve the efficiency and consistency of the database.
- **Ensuring Data Consistency:** Normalization helps in ensuring that the data in the database is consistent and accurate. By eliminating redundancy, normalization helps in preventing inconsistencies and contradictions that can arise due to different versions of the same data.
- **Simplification of Data Management:** Normalization simplifies the process of managing data in a database. By breaking down a complex data structure into simpler tables, normalization makes it easier to manage the data, update it, and retrieve it.
- **Improved Database Design:** Normalization helps in improving the overall design of the database. By organizing the data in a structured and systematic way, normalization makes it easier to design and maintain the database. It also makes the database more flexible and adaptable to changing business needs.
- **Standardization:** Normalization helps in standardizing the data in the database. By organizing the data into tables and defining relationships between them, normalization helps in ensuring that the data is stored in a consistent and uniform manner.

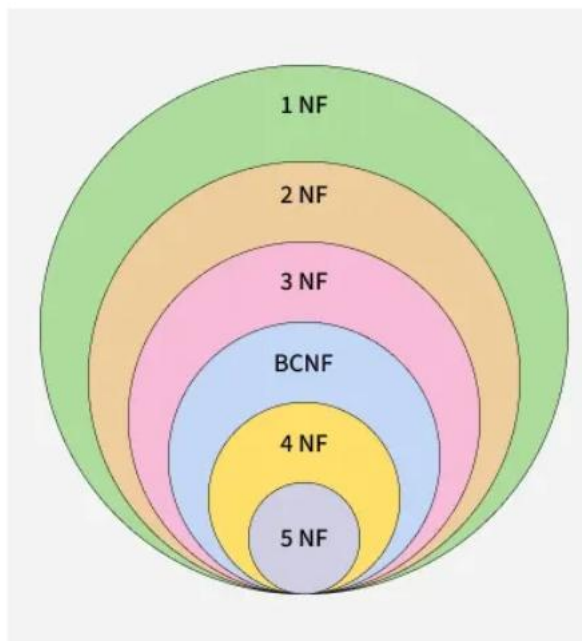
## Normal Forms in DBMS

Normal forms are a set of progressive rules (or design checkpoints) for relational schemas that reduce redundancy and prevent data anomalies. Each normal form - 1NF, 2NF, 3NF, BCNF, 4NF, 5NF - is stricter than the previous one: meeting a higher normal form implies the lower ones are satisfied. Think of them as layers of cleanliness for your tables: the deeper you go, the fewer redundancy and integrity problems you'll have.

### Benefits of using Normal Forms

- Reduce duplicate data and wasted storage.
- Prevent insert, update, and delete anomalies.
- Improve data consistency and integrity.
- Make the schema easier to maintain and evolve.

The Diagram below shows the hierarchy of database normal forms. Each inner circle represents a stricter level of normalization, starting from 1NF (basic structure) to 5NF (most refined). As you move inward, data redundancy reduces and data integrity improves. Each level builds upon the previous one to ensure a cleaner and more efficient database design.



### Common Challenges of Over-Normalization

While normalization is a powerful tool for optimizing databases, it's important not to over-normalize your data. Excessive normalization can lead to:

- **Complex Queries:** Too many tables may result in multiple joins, making queries slow and difficult to manage.

- **Performance Overhead:** Additional processing required for joins in overly normalized databases may hurt performance, especially in large-scale systems.

In many cases, denormalization (combining tables to reduce the need for complex joins) is used for performance optimization in specific applications, such as reporting systems.

### When to Use Normalization and Denormalization

- **Normalization** is best suited for transactional systems where data integrity is paramount, such as banking systems and enterprise applications.
- **Denormalization** is ideal for read-heavy applications like data warehousing and reporting systems where performance and query speed are more critical than data integrity.

### Applications of Normal Forms in DBMS

- **Ensures Data Consistency:** Prevents data anomalies by ensuring each piece of data is stored in one place, reducing inconsistencies.
- **Reduces Data Redundancy:** Minimizes repetitive data, saving storage space and avoiding errors in data updates or deletions.
- **Improves Query Performance:** Simplifies queries by breaking large tables into smaller, more manageable ones, leading to faster data retrieval.
- **Enhances Data Integrity:** Ensures that data is accurate and reliable by adhering to defined relationships and constraints between tables.
- **Easier Database Maintenance:** Simplifies updates, deletions, and modifications by ensuring that changes only need to be made in one place, reducing the risk of errors.
- **Facilitates Scalability:** Makes it easier to modify, expand, or scale the database structure as business requirements grow.
- **Supports Better Data Modeling:** Helps in designing databases that are logically structured, with clear relationships between tables, making it easier to understand and manage.
- **Reduces Update Anomalies:** Prevents issues like insertion, deletion, or modification anomalies that can arise from redundant data.
- **Improves Data Integrity and Security:** By reducing unnecessary data duplication, normal forms help ensure sensitive information is securely and correctly maintained.
- **Optimizes Storage Efficiency:** By organizing data into smaller tables, storage is used more efficiently, reducing the overhead for large databases
-

## First Normal Form (1NF)

First Normal Form (1NF) ensures that the structure of a database table is organized in a way that makes it easier to manage and query.

- A relation is in first normal form if every attribute in that relation is single-valued attribute or it does not contain any composite or multi-valued attribute.
- It is the first and essential step in reduce redundancy, improve data integrity and reducing anomalies in relational database design.

A relation (table) is said to be in First Normal Form (1NF) if:

- All the attributes (columns) contain only atomic (indivisible) values.
- Each column contains values of a single type.
- Each record (row) is unique, meaning it can be identified by a primary key.
- There are no repeating groups or arrays in any row.

### Rules for First Normal Form (1NF) in DBMS

To follow the First Normal Form (1NF) in a database, these simple rules must be followed:

#### Every Column Should Have Single Values

Each column in a table must contain only one value in a cell. No cell should hold multiple values. If a cell contains more than one value, the table does not follow 1NF.

- **Example:** A table with columns like [Writer 1], [Writer 2], and [Writer 3] for the same book ID is not in 1NF because it repeats the same type of information (writers). Instead, all writers should be listed in separate rows.

#### All Values in a Column Should Be of the Same Type

Each column must store the same type of data. You cannot mix different types of information in the same column.

- **Example:** If a column is meant for dates of birth (DOB), you cannot use it to store names. Each type of information should have its own column.

#### Every Column Must Have a Unique Name

Each column in the table must have a unique name. This avoids confusion when retrieving, updating, or adding data.

- **Example:** If two columns have the same name, the database system may not know which one to use.

#### The Order of Data Doesn't Matter

In 1NF, the order in which data is stored in a table doesn't affect how the table works. You can organize the rows in any way without breaking the rules.

**Example:**

Consider the below COURSES Relation :

**COURSES Table**

| ID | Name | Courses |
|----|------|---------|
| 1  | A    | c1, c2  |
| 2  | E    | c3      |
| 3  | M    | c2, c3  |

In the above table, Courses has a multi-valued attribute, so it is not in 1NF. To make the table in 1NF we have to remove the multivalued attributes from the table as given below:

**COURSES Table**

| ID | Name | Courses |
|----|------|---------|
| 1  | A    | c1      |
| 1  | A    | c2      |
| 2  | E    | c3      |
| 3  | M    | c2      |
| 3  | M    | c3      |

1NF

Now the table is in 1NF as there is no multi-valued attribute present in the table.

## Second Normal Form (2NF)

Second Normal Form (2NF) is based on the concept of fully functional dependency. It is a way to organize a database table so that it reduces redundancy and ensures data consistency. Fully Functional Dependency means a non-key attribute depends on the **entire** primary key, not just part of it.

For a table to be in 2NF, it must first meet the following requirements

**1. Meet 1NF Requirements:** The table must first satisfy **First Normal Form (1NF)**, meaning:

- All columns contain single, indivisible values.
- No repeating groups of columns.

**2. Eliminate Partial Dependencies:** A partial dependency occurs when a non-prime attribute (not part of the candidate key) depends only on a part of a composite primary key, rather than the entire key.

By ensuring these steps, a table in 2NF is more efficient and less prone to errors during updates, inserts, and deletes.

### What is Partial Dependency?

The FD (functional dependency)  $A \rightarrow B$  happens to be a partial dependency if B is functionally dependent on A, and also B can be determined by any other proper subset of A.

In other words, if you have a composite key (a primary key made up of more than one attribute), and an attribute depends on only a subset of that composite key, rather than the entire key, that is considered a partial dependency.

A partial dependency would occur whenever a non-prime attribute depends functionally on a part of the given candidate key.

**Example:**

## StaffBranch

| staffNo | sName   | position   | salary | branchNo |
|---------|---------|------------|--------|----------|
| SL21    | Tannu   | Manager    | 30000  | B005     |
| SG37    | Leela   | Assistant  | 12000  | B003     |
| SG14    | Henry   | Supervisor | 18000  | B003     |
| SA9     | Suresh  | Assistant  | 9000   | B007     |
| SG5     | Anthony | Manager    | 24000  | B003     |
| SL41    | Julie   | Assistant  | 9000   | B005     |

*StaffBranch Relation*

In the given relation StaffBranch, we have the functional dependency:

- $\text{staffNo, sName} \rightarrow \text{branchNo}$ .

This means that the combination of staffNo and sName determines branchNo.

BranchNo is also functionally dependent on a subset of the composite key, specifically staffNo. This means that branchNo can be determined by just staffNo.

- $\text{staffNo} \rightarrow \text{branchNo}$

This is a partial dependency because branchNo depends on only a part of the composite key (staffNo, sName), not the entire key.

### Example of Second Normal Form (2NF)

Consider a table storing information about students, courses, and their fees:

## COURSES Table

| STUD_NO | COURSE_NO | COURSE_FEE |
|---------|-----------|------------|
| 1       | C1        | 1000       |
| 2       | C2        | 1500       |
| 1       | C4        | 2000       |
| 4       | C3        | 1000       |
| 4       | C1        | 1000       |
| 2       | C5        | 2000       |

- There are many courses having the same course fee. Here, COURSE\_FEE cannot alone decide the value of COURSE\_NO or STUD\_NO.
- COURSE\_FEE together with STUD\_NO cannot decide the value of COURSE\_NO.
- COURSE\_FEE together with COURSE\_NO cannot decide the value of STUD\_NO.
- The candidate key for this table is {STUD\_NO, COURSE\_NO} because the combination of these two columns uniquely identifies each row in the table.
- COURSE\_FEE is a **non-prime attribute** because it is not part of the candidate key {STUD\_NO, COURSE\_NO}.
- But, COURSE\_NO → COURSE\_FEE, i.e., COURSE\_FEE is dependent on COURSE\_NO, which is a proper subset of the candidate key.
- Therefore, Non-prime attribute COURSE\_FEE is dependent on a proper subset of the candidate key, which is a partial dependency and so this relation is not in 2NF.

**In 2NF, we eliminate such dependencies by breaking the table into two separate tables:**

1. A table that links students and courses.
2. A table that stores course fees.

| STUDENT_COURSES Table |           |
|-----------------------|-----------|
| STUD_NO               | COURSE_NO |
| 1                     | C1        |
| 2                     | C2        |
| 1                     | C4        |
| 4                     | C3        |
| 4                     | C1        |
| 2                     | C5        |

| COURSES Table |            |
|---------------|------------|
| COURSE_NO     | COURSE_FEE |
| C1            | 1000       |
| C2            | 1500       |
| C4            | 2000       |
| C3            | 1000       |
| C5            | 2000       |

**Now, each table is in 2NF:**

- The Course Table ensures that **COURSE\_FEE** depends only on **COURSE\_NO**.
- The Student-Course Table ensures there are no partial dependencies because it only relates students to courses.

Now, the **COURSE\_FEE** is no longer repeated in every row, and each table is free from partial dependencies. This makes the database **more efficient** and **easier to maintain**.

## Limitations of Second Normal Form (2NF)

While Second Normal Form (2NF) addresses partial dependencies and helps reduce redundancy, it has some limitations:

**1. Doesn't Handle Transitive Dependencies:** 2NF ensures that non-prime attributes are fully dependent on the entire primary key, but it doesn't address transitive dependencies. In a transitive dependency, an attribute depends on another non-key attribute.

For example, if  $A \rightarrow B$  and  $B \rightarrow C$ , then A indirectly determines C. This can lead to further redundancy and anomalies.

**2. Doesn't Ensure Optimization:** Although 2NF eliminates partial dependencies, it may still leave some redundancy in the data, particularly when dealing with larger and more complex datasets. It doesn't guarantee the most efficient or optimized structure for a database.

**3. Complexity in Handling Multi-Attribute Keys:** When dealing with composite primary keys (keys made of multiple attributes), ensuring full dependency can still lead to a complex design. A further step of normalization (Third Normal Form or 3NF) is required to resolve transitive dependencies and achieve better data organization.

**4. Not Sufficient for Some Use Cases:** While 2NF is useful for reducing redundancy in some situations, in real-world applications where data integrity and efficiency are crucial, additional normalization (like 3NF) might be needed to address more complex dependencies and optimize data storage and retrieval.

## Third Normal Form (3NF)

The Third Normal Form (3NF) builds on the First (1NF) and Second (2NF) Normal Forms. Achieving 3NF ensures that the database structure is free of transitive dependencies, reducing the chances of data anomalies. Even though tables in 2NF have reduced redundancy compared to 1NF, they may still encounter issues like **update anomalies**.

A relation is in **Third Normal Form (3NF)** if it satisfies the following two conditions:

1. It is in **Second Normal Form (2NF)**: This means the table has no partial dependencies (i.e., no non-prime attribute is dependent on a part of a candidate key).
2. There is **no transitive dependency for non-prime attributes**: In simpler terms, no non-key attribute should depend on another non-key attribute. Instead, all non-key attributes should depend directly on the primary key.

### Understanding Transitive Dependency

To fully grasp 3NF, it's essential to understand **transitive dependency**. A **transitive dependency** occurs when one non-prime attribute depends on another non-prime attribute rather than depending directly on the primary key. This can create redundancy and inconsistencies in the database.

For example, if we have the following relationship between attributes:

- **A → B** (A determines B)
- **B → C** (B determines C)

This means that **A** indirectly determines **C** through **B**, creating a **transitive dependency**. 3NF eliminates these transitive dependencies to ensure that non-key attributes are directly dependent only on the primary key.

### Conditions for a Table to be in 3NF

A table is in **Third Normal Form (3NF)** if, for every non-trivial functional dependency  $X \rightarrow Y$ , at least one of the following holds:

- **X is a superkey**: This means that the attribute(s) on the left-hand side of the functional dependency (X) must be a superkey (a key that uniquely identifies a tuple in the table).
- **Y is a prime attribute**: This means that every element of the attribute set Y must be part of a candidate key (i.e., a prime attribute).

### Example 1: Third Normal Form (3NF)

Consider the following relation for a **Candidate** table with the following attributes and functional dependencies:

### CANDIDATE Table

| CAND_NO | CAND_NAME | CAND_STATE  | CAND_COUNTRY | CAND_AGE |
|---------|-----------|-------------|--------------|----------|
| 1       | Amayra    | West Bengal | India        | 18       |
| 2       | Rihaan    | Haryana     | India        | 17       |
| 3       | Manya     | Haryana     | India        | 19       |

#### 1. Functional dependency Set:

The set of functional dependencies is as follows:

- **CAND\_NO → CAND\_NAME**
- **CAND\_NO → CAND\_STATE**
- **CAND\_STATE → CAND\_COUNTRY**
- **CAND\_NO → CAND\_AGE**

#### 2. Determining the Candidate Key:

The **candidate key** for this relation is {**CAND\_NO**}, since **CAND\_NO** uniquely identifies all other attributes in the table.

#### 3. Identifying Transitive Dependency:

The issue here arises from the **transitive dependency** between **CAND\_NO** and **CAND\_COUNTRY**:

- **CAND\_NO → CAND\_STATE**
- **CAND\_STATE → CAND\_COUNTRY**

This means that **CAND\_COUNTRY** is transitively dependent on **CAND\_NO** via **CAND\_STATE**, which violates the **Third Normal Form (3NF)** rule that states that no non-prime attribute (non-key attribute) should be transitively dependent on the primary key.

#### Converting the Relation into 3NF

To remove the transitive dependency and ensure the relation is in **3NF**, we decompose the original **CANDIDATE** relation into two separate relations:

1. **CANDIDATE**: This will store information about the candidates, including their **CAND\_NO**, **CAND\_NAME**, **CAND\_STATE**, and **CAND\_AGE**:  
 $\text{CANDIDATE (CAND\_NO, CAND\_NAME, CAND\_STATE, CAND\_AGE)}$
2. **STATE\_COUNTRY**: This relation will store information about the states and their respective countries:  
 $\text{STATE\_COUNTRY (CAND\_STATE, CAND\_COUNTRY)}$

#### Final Decomposed Relations:

1. **CANDIDATE (CAND\_NO, CAND\_NAME, CAND\_STATE, CAND\_AGE)**
2. **STATE\_COUNTRY (CAND\_STATE, CAND\_COUNTRY)**

#### Why This Decomposition Works:

- The **CANDIDATE** relation now no longer has a transitive dependency. **CAND\_STATE** no longer determines **CAND\_COUNTRY** within this relation.
- The **STATE\_COUNTRY** relation handles the **CAND\_STATE** → **CAND\_COUNTRY** dependency separately, ensuring that all data is now organized in a way that satisfies **3NF**.

| CANDIDATE Table |           |             |          |
|-----------------|-----------|-------------|----------|
| CAND_NO         | CAND_NAME | CAND_STATE  | CAND_AGE |
| 1               | Amayra    | West Bengal | 18       |
| 2               | Rihaan    | Haryana     | 17       |
| 3               | Manya     | Haryana     | 19       |

| STATE_COUNTRY Table |              |
|---------------------|--------------|
| CAND_STATE          | CAND_COUNTRY |
| West Bengal         | India        |
| Haryana             | India        |

#### Example 2: Relation R(A, B, C, D, E) (SELF STUDY)

Consider the relation **R(A, B, C, D, E)** with the following functional dependencies:

**A → BC**

**CD → E**

**B → D**

**E → A**

#### Step 1: Identify Candidate Keys

A **candidate key** is a minimal set of attributes that can uniquely identify a tuple (row) in the relation. In this case, the possible candidate keys for the relation are **{A, E, CD, BC}**. This means that any of these sets of attributes can uniquely identify all other attributes in the relation.

#### Step 2: Check Functional Dependencies

Let's analyze the given functional dependencies:

1.  $A \rightarrow BC$ : This means that knowing **A** allows us to determine both **B** and **C**.
2.  $CD \rightarrow E$ : Knowing **CD** allows us to determine **E**.
3.  $B \rightarrow D$ : Knowing **B** allows us to determine **D**.
4.  $E \rightarrow A$ : Knowing **E** allows us to determine **A**.

We observe that all attributes on the right-hand side of the functional dependencies are **prime attributes** (i.e., they are part of some candidate key). This means no non-prime attribute is dependent on another non-prime attribute (which would be a **transitive dependency**).

### Step 3: Check for Transitive Dependencies

In **3NF**, a relation must be free of transitive dependencies, where a non-prime attribute depends on another non-prime attribute indirectly via the primary key.

- Here,  $A \rightarrow BC$  and  $B \rightarrow D$ , so **B** is a non-prime attribute that determines **D**, and **A** determines **B**. However, since **B** is part of a candidate key, this does not introduce a transitive dependency.
- $E \rightarrow A$  and  $A \rightarrow BC$ , meaning **E** determines **A**, and then **A** determines **B** and **C**. Again, no transitive dependency is formed because **A** is part of a candidate key.

Since there are no transitive dependencies, the relation **R** satisfies the condition of **3NF**.

### Step 4: Conclusion

Relation **R(A, B, C, D, E)** is already in **Third Normal Form (3NF)** because:

- There are no transitive dependencies.
- All non-prime attributes are functionally dependent only on candidate keys.

### Why is 3NF Important?

**1. Eliminates Redundancy:** **3NF** helps to remove unnecessary duplication of data by ensuring that non-prime attributes (attributes not part of any candidate key) depend directly on the primary key, not on other non-prime attributes.

**2. Prevents Anomalies:** A table in **3NF** is free from common anomalies such as:

- **Insertion Anomaly:** The inability to insert data without having to insert unwanted or redundant data.
- **Update Anomaly:** The need to update multiple rows of data when a change occurs in one place.

- **Deletion Anomaly:** The unintended loss of data when a record is deleted.

**3. Preserves Functional Dependencies:** **3NF** ensures that all functional dependencies are preserved, meaning that the relationships between attributes are maintained.

**4. Lossless Decomposition:** When decomposing a relation to achieve **3NF**, the decomposition should be **lossless**, meaning no information is lost in the process of normalization.

## Boyce-Codd Normal Form (BCNF)

Boyce-Codd Normal Form (BCNF) is an advanced version of 3NF used to reduce redundancy in databases. It ensures that for every functional dependency, the left side must be a superkey. This helps create cleaner and more consistent database designs, especially when there are multiple candidate keys.

| Fourth Normal Form |            |                 |
|--------------------|------------|-----------------|
| Course             | Instructor | TextBook_Author |
| Management         | X          | Churchil        |
| Management         | Y          | Peters          |
| Management         | Z          | Peters          |
| Finance            | A          | Weston          |
| Finance            | A          | Gilbert         |

| Course     |            | TextBook Author |            |
|------------|------------|-----------------|------------|
| Course     | Instructor | Course          | Instructor |
| Management | X          | Management      | Churchil   |
| Management | Y          | Management      | Peters     |
| Management | Z          | Finance         | Weston     |
| Finance    | A          | Finance         | Gilbert    |

4th Normal Form

**Example:** Consider the database table of a class that has two relations R1 contains student ID(SID) and student name (SNAME) and R2 contains course id(CID) and course name (CNAME).

**Table R:**

| Course     | Instructor | TextBook_Author |
|------------|------------|-----------------|
| Management | X          | Churchill       |
| Management | Y          | Peters          |
| Management | Z          | Peters          |
| Finance    | A          | Weston          |

| Course  | Instructor | TextBook_Author |
|---------|------------|-----------------|
| Finance | A          | Gilbert         |

**Problem:**

- Each Course has multiple Instructors.
- Each Course has multiple TextBook\_Author.
- But Instructor and TextBook\_Author are not related to each other.
- This causes repetition of combinations, violating 4NF.

**Solution:** To remove the MVDs and bring the relation to Fourth Normal Form, we split the original table into two separate tables, each handling one multivalued dependency. This improves data integrity and removes redundancy.

**Table R1:**

| Course     | Instructor |
|------------|------------|
| Management | X          |
| Management | Y          |
| Management | Z          |
| Finance    | A          |

*This table shows which instructor teaches which course.*

**Table R2:**

| Course     | TextBook_Author |
|------------|-----------------|
| Management | Churchill       |

| Course     | TextBook_Author |
|------------|-----------------|
| Management | Peters          |
| Finance    | Weston          |
| Finance    | Gilbert         |

**Result:** Now, the 4NF is Achieved

### Rules for BCNF

- **Rule 1:** The table should be in the 3rd Normal Form.
- **Rule 2:** X should be a super-key for every functional dependency (FD)  $X \rightarrow Y$  in a given relation.

**Note:** To test whether a relation is in BCNF, we identify all the determinants and make sure that they are candidate keys.

### Key Notes:

1. To verify BCNF, identify all determinants (left side of FDs) and check whether each is a candidate key.
2. If a relation is in BCNF, it is automatically in 3NF, 2NF, and 1NF as well.

The normal forms become stricter as we move from  $1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$ :

- 1NF: Each field must hold atomic (indivisible) values.
- 2NF: No partial dependency on a primary key.
- 3NF: No transitive dependency on a primary key.
- BCNF: Every determinant must be a candidate key.

This progression ensures better structure and removes redundancy at each level.

### Why Do We Need BCNF?

- 2NF and 3NF may allow anomalies if a functional dependency exists where the determinant is not a superkey.

- BCNF handles edge cases where 3NF fails to remove all redundancy, especially in tables with multiple candidate keys.
- Prevents update, insert, and delete anomalies by ensuring every determinant is a superkey.
- Makes database design more robust and easier to maintain over time.
- Improves data consistency and clarity by removing hidden or indirect dependencies.

We are going to discuss some basic examples which let you understand the properties of BCNF. We will discuss multiple examples here.

### Example 1

Consider a relation R with attributes (student, teacher, subject).

| R       |         |          |
|---------|---------|----------|
| Student | Teacher | Subject  |
| Shayan  | R.Das   | Database |
| Shayan  | K.Raman | C        |
| Tahira  | R.Das   | Database |
| Tahira  | N.Gupta | C        |

FD: { (student, Teacher) → subject, (student, subject) → Teacher, (Teacher) → subject }

- Candidate keys are (student, teacher) and (student, subject).
- The above relation is in 3NF (since there is no transitive dependency). A relation R is in BCNF if for every non-trivial FD  $X \rightarrow Y$ , X must be a key.
- The above relation is not in BCNF, because in the FD (teacher → subject), teacher is not a key. This relation suffers with anomalies –
- For example, if we delete the student Tahira, we will also lose the information that N.Gupta teaches C. This issue occurs because the teacher is a determinant but not a candidate key.

| R1      |          | R2      |         |
|---------|----------|---------|---------|
| Teacher | Subject  | Student | Teacher |
| R.Das   | Database | Shayan  | R.Das   |
| K.Raman | C        | Shayan  | K.Raman |
| N.Gupta | C        | Tahira  | R.Das   |
|         |          | Tahira  | N.Gupta |

R is divided into two relations R1(Teacher, Subject) and R2(Student, Teacher).

For more, refer to BCNF in DBMS.

### How to Satisfy BCNF?

For satisfying this table in BCNF, we have to decompose it into further tables. Here is the full procedure through which we transform this table into BCNF. Let us first divide this main table into two tables **Stu\_Branch** and **Stu\_Course** Table.

#### Stu\_Branch Table

| Stu_ID | Stu_Branch                              |
|--------|-----------------------------------------|
| 101    | Computer Science & Engineering          |
| 102    | Electronics & Communication Engineering |

Candidate Key for this table: **Stu\_ID**.

#### Stu\_Course Table

| Stu_Course | Branch_Number | Stu_Course_No |
|------------|---------------|---------------|
| DBMS       | B_001         | 201           |

| Stu_Course           | Branch_Number | Stu_Course_No |
|----------------------|---------------|---------------|
| Computer Networks    | B_001         | 202           |
| VLSI Technology      | B_003         | 401           |
| Mobile Communication | B_003         | 402           |

Candidate Key for this table: **Stu\_Course**.

#### Stu\_Enroll Table

| Stu_ID | Stu_Course_No |
|--------|---------------|
| 101    | 201           |
| 101    | 202           |
| 102    | 401           |
| 102    | 402           |

Candidate Key for this table: **{Stu\_ID, Stu\_Course\_No}**.

After decomposing into further tables, now it is in BCNF, as it is passing the condition of Super Key, that in functional dependency  $X \rightarrow Y$ , X is a Super Key.